

Page 1: Introduction to Python

Python is a high-level programming language known for readable syntax and a large ecosystem of libraries. It is commonly used for automation, data analysis, web development, artificial intelligence, and education.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 1 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 2: Variables and Data Types

Variables store values that a program can use later. Common Python data types include integers, floating-point numbers, strings, booleans, lists, tuples, sets, and dictionaries.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 2 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 3: Conditional Statements

Conditional statements allow a program to make decisions. The `if`, `elif`, and `else` keywords can be combined to execute different blocks of code depending on whether conditions are true or false.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 3 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 4: Loops

Loops repeat a block of instructions. Python provides for loops and while loops. Loops are useful when processing collections, repeating calculations, or performing an operation until a condition changes.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 4 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 5: Functions

Functions organize reusable pieces of code. A function can receive parameters, perform operations, and return a result. Good functions usually have clear names and a focused purpose.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 5 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 6: Lists

Lists are ordered and mutable collections. They can contain values of different types and support indexing, slicing, adding elements, removing elements, and iteration.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 6 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 7: Dictionaries

Dictionaries store data as key-value pairs. They are useful when information needs to be accessed by a meaningful key rather than by a numeric position.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 7 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 8: Strings

Strings represent text. Python provides many string operations, including concatenation, slicing, searching, replacing text, changing case, and splitting text into pieces.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 8 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 9: File Handling

Python can read from and write to files. File handling is useful for processing documents, logs, configuration files, datasets, and other persistent information.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 9 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 10: Exceptions

Exceptions are events that interrupt normal program execution. try, except, else, and finally blocks can be used to handle errors gracefully and keep programs robust.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 10 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 11: Modules

A module is a Python file containing code that can be imported into another program. Modules help separate functionality and make projects easier to organize and maintain.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 11 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 12: Object-Oriented Programming

Object-oriented programming organizes software around classes and objects. Classes can define attributes and methods that describe the data and behavior of objects.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 12 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 13: Working with JSON

JSON is a common text format for structured data. Python's json module can convert between Python objects and JSON strings or files.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 13 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 14: APIs and Data

An API provides a structured way for programs to communicate. Applications often receive JSON data from APIs and then transform that information for users or other systems.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 14 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 15: Regular Expressions

Regular expressions provide patterns for finding and validating text. They are useful for tasks such as extracting numbers, checking formats, and locating repeated text patterns.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 15 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 16: Data Cleaning

Data cleaning involves detecting missing, duplicated, inconsistent, or incorrectly formatted information. Clean data is easier to analyze and produces more reliable results.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 16 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 17: Pandas Basics

Pandas provides data structures and tools for working with tabular data. DataFrames are especially useful for filtering, grouping, transforming, and summarizing datasets.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 17 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 18: NumPy Basics

NumPy provides efficient numerical arrays and mathematical operations. It is widely used as a foundation for scientific computing and data analysis in Python.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 18 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 19: Introduction to Machine Learning

Machine learning uses data to build models that can identify patterns and make predictions. Common categories include supervised, unsupervised, and reinforcement learning.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 19 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 20: Natural Language Processing

Natural language processing focuses on working with human language using computers. Typical tasks include text classification, summarization, search, translation, and question answering.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 20 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 21: PDF Documents

PDF files preserve document layout across many systems. They can contain text, images, tables, links, metadata, and other elements. Python libraries can extract and process PDF content.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 21 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 22: PyPDF and PDF Loading

PyPDF is a Python library for reading and manipulating PDF documents. A PDF loader built around PyPDF can extract pages and expose their text as documents for downstream processing.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 22 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 23: Document Chunking

Long documents are often divided into smaller chunks before being processed by language models. Chunk size and overlap can affect retrieval quality and the amount of context available.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 23 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 24: Embeddings and Vector Search

Embeddings represent text as numerical vectors. Similar pieces of text tend to have related vector representations, making embeddings useful for semantic search and retrieval systems.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 24 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.

Page 25: Retrieval-Augmented Generation

Retrieval-augmented generation combines document retrieval with language generation. Relevant chunks are retrieved from a knowledge source and supplied as context to a language model.

This practice page is designed for PDF loading and text extraction experiments. You can test page counting, metadata inspection, page-by-page extraction, searching for keywords, and splitting the extracted content into chunks. Page 25 contains a small amount of intentionally structured text so that you can verify whether your loader preserves page boundaries correctly.

Practice idea: load this PDF, inspect the returned documents, print the first few characters from each page, and check whether the page number and heading can be detected. Then experiment with different chunk sizes and overlap values.